

Workshop 9: Dense matrix-matrix multiplication

1 Introduction

In this workshop we will investigate the performance of dense matrix-matrix multiplication on ISCA. We will compare the performance of the BLAS DGEMM¹ routine from the Intel Math Kernel Library with a simple implementation using for loops.

2 Getting set up

- Start by logging in to ISCA.
- The example programs for this workshop are provided as a tar file. There are two versions of the tar file (one for ECM3446 and one for ECMM461) Take a copy of the tar file with the correct module code by running either

```
cp /gpfs/ts0/projects/Research_Project-IscaTraining/HPC/workshop9_ecm3446.tar .
```

or

```
cp /gpfs/ts0/projects/Research_Project-IscaTraining/HPC/workshop9_ecmm461.tar .
```

where the dot at the end of the command causes the file to be copied into the current working directory. When you first log in to ISCA the current working directory is your home directory, which is mounted on the log in nodes and all the compute nodes. There should be sufficient space in your home directory for all the workshops on this module.

- Unpack the tar file.

This will create a directory called workshop9 which contains the example program for this workshop.

3 Test programs: for loops and DGEMM

In this section we will run two small test programs which perform matrix-matrix multiplication using either nested for loops or DGEMM. The first program we will look at is the version which uses for loops.

- Start by loading the Intel Cluster Toolkit module by running the command

```
module load intel/2023b
```

- Now run the below to load the OpenBLAS module, which gives access to dense matrix-matrix multiplication operation.

```
module load OpenBLAS/0.3.24-GCC-13.2.0
```

- Open the Makefile using `nano` and update as below and save changes.

```
CC=icc
matmul: matmul.c
    $(CC) -o matmul matmul.c
matmul_big: matmul_big.c
    $(CC) -fopenmp -o matmul_big matmul_big.c
dgemm: dgemm.c
    $(CC) -qmkl -o dgemm dgemm.c
dgemm_big: dgemm_big.c
    $(CC) -qmkl -o dgemm_big dgemm_big.c
```

- The for loop matrix-matrix multiplication program is called `matmul.c`. Compile the program using the supplied Makefile by running:

```
cd workshop9
```

```
make matmul
```

When the program runs it multiplies two matrices (**A** and **B**) and adds the result to a matrix **C** ($C := C + AB$). The matrices are all 3×3 and **C** is initially the zero matrix. At the end of the program the matrices **A**, **B** and **C** are printed out.

- This is a very small test program, so it does not need to be run in the queue. Run the program directly from the command line

```
./matmul
```

and check that the results are as expected.

The second version of the program multiplies the matrices using the BLAS routine DGEMM. DGEMM calculates $C \leftarrow \alpha AB + \beta C$. In the example program $\alpha = 1.0$ and $\beta = 0.0$ and the matrices are the same 3×3 matrices used in the previous example.

- Build the DGEMM test program by running

```
make dgemm
```

- This is also a small program which can be run directly from the command line. Run the DGEMM test program

```
./dgemm
```

and check that the results match the previous results.

4 Comparing performance

In this section we will compare the performance of the two implementations of matrix-matrix multiplication. For these tests the problem size needs to be increased and the program run in the queue using the supplied job scripts.

The program `matmul_big.c` is a modified version of `matmul.c` which has been parallelised using OpenMP. The size of the matrices to use is read in as a command line argument and this value is supplied in the job script. The program records the time taken to multiply the matrices and uses this to calculate the flops (floating point operations per second) achieved.

- Compile the OpenMP version of the matmul program using make:

```
make matmul_big
```

- Now update the queue to 'pq' and load the intel and the OpenBLAS modules in the job script run_matmul_big.sh, as below and save changes. You can use nano to edit the job script.

```
#!/bin/bash
# Number of nodes required
#SBATCH --nodes=1
# Number of tasks per node
#SBATCH --ntasks-per-node=1
# Number of CPUs per task
#SBATCH --cpus-per-task=16
# Maximum time the job will run for (HH:MM:SS)
#SBATCH --time=5:00
# Name of the job
#SBATCH --job-name=Hello_OpenMP
# Queue for the job to run in: pq
# Specify the account code and reservation
#SBATCH -A Research_Project-IscaTraining

# Unload all existing modules
module purge
# Load the Intel Cluster Toolkit module used to compile the program
module load intel/2023b
module load OpenBLAS/0.3.24-GCC-13.2.0

# Set the number of OpenMP threads
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
echo "Threads: ${OMP_NUM_THREADS}"

# Pin threads to cores
export KMP_AFFINITY=compact

# Run the program
./matmul_big 8000

# End of file
```

- Now submit the job.

```
sbatch run_matmul_big.sh
```

This will run the program using 16 OpenMP threads.

Question 1: How long does the matmul program take to multiply two 8000 x 8000 matrices?

The DGEMM function in the Math Kernel Library is multi-threaded so we can run the DGEMM version of the program using 16 threads and compare the results with matmul_big.c. The program dgemm_big.c reads in the size of a matrix as a command line argument and calculates matrixmatrix multiplication using DGEMM. It times the call to DGEMM and uses this to calculate the flops achieved based on the size of the matrices and the time taken by the DGEMM call.

- Build the corresponding DGEMM version (dgemm_big.c) of the program.

```
make dgemm_big
```

- Update the queue to 'pq' and load the intel and OpenBLAS modules in the job script run_dgemm_big.sh (as you did previously) and submit the job to the queue.

```
sbatch run_dgemm_big.sh
```

Question 2: How long does the DGEMM program take to multiply two 8000 x 8000 matrices and how much faster it is than the matmul program?

5 Scaling of DGEMM program

In this section we will see how well the DGEMM program scales. The job script `run_dgemm_multi.sh` runs the `dgemm_big` program with 2,4,8,12 and 16 threads.

- Update the queue to 'pq' and load the intel and OpenBLAS modules in the job script `run_dgemm_multi.sh` (as you did previously) and save changes.
- Submit the job to the queue to run the scaling test

```
sbatch run_dgemm_multi.sh
```

Question: 3 What fraction of peak performance does the DGEMM program achieve? The processor can deliver 16 operations per cycle and the base clock frequency is 2.60 GHz.

6 Changing the matrix size

In this last section we will investigate the effect of the matrix size on the performance. The job script `run_dgemm_var.sh` runs the DGEMM program with different size matrices using 8 threads.

- Update the queue to 'pq' and load the intel and OpenBLAS modules in the job script `run_dgemm_var.sh` (as you did previously) and save changes.
- Submit the to the queue to run the test

```
sbatch run_dgemm_var.sh
```

Question: 4 How does the measured performance (flops) depend on the size of the matrix and is this what you would expect?